COMP 2322 COMPUTER NETWORKING

Multi-Thread Web Server Project

Project Report

Name	Samuel W. Ha
Student ID	25142247X
Course Name	COMP2322 Computer Networking
Submission	April 26, 2026
GitHub	https://github.com/SMLG120/Multi-thread-Web-Server

1. Design & Implementation Summary:

The project implements a multi-threaded HTTP/1.1 web server in Python using the built-in socket module — without any higher-level libraries such as http.server.HTTPServer. The server can handle multiple simultaneous client connections, serve both text and binary files, and correctly implement five HTTP response status codes.

1.1 Architecture overview:

The server is organized into four distinct layers.

1) The Webserver (main thread) — which creates the TCP socket, binds to 127.0.0.1:8080, and runs an infinite accept-loop. Every accepted connection is handed to a new daemon thread immediately, so the main thread never blocks
2) Client Handler(one thread per connection) — owns the full request/response lifecycle for a single client. It loops for keep-alive connections and exits after one response for non-persistent connections.
3) Utility modules — request_parser.py, repsonse_builder.py and logger.py are stateless helpers imported by Client Handler. The only shared mutable state is the log file, protected by a threading Lock.
4) Testing — which includes a bash script (run_tests.sh) to test the requests and responses, located under the test folder.

1.2 Module breakdown:

File	Responsibility
`server.py`	Entry point. WebServer accept-loop + ClientHandler thread spawning.
`utils/request_parser.py`	Parses raw bytes into HTTPRequest dataclass. Raises MalformedRequestError for 400 cases.
`utils/response_builder.py`	Builds HTTP response bytes for 200, 304, 400, 403, 404. Handles GET vs HEAD, MIME types, and Last-Modified.
`utils/logger.py`	Thread-safe append logger. One tab-delimited line per request is written to logs/server.log.
`static/`	Document root. All files served to clients must reside here.
`logs/server.log`	Auto-created request log. One entry per HTTP transaction.
`test/run_tests.sh`	Bash script for testing case scenarios.

1.3 Multi-threading model:

Each call to server_socket.accept() blocks until a client connects and returns a new dedicated socket. A threading.Thread is created with that socket as an argument and started immediately in the server.py file. Setting daemon=True ensures threads are cleaned up automatically when the main process exits, preventing zombie threads.

```
conn, addr = server_socket.accept()
handler = ClientHandler(conn, addr, self.logger)
thread  = threading.Thread(target=handler.handle, daemon=True)
thread.start()   # main thread immediately loops back to accept()
```

1.4 HTTP request parsing

The parser reads bytes from the socket in a loop until the **`\r\n\r\n`** end-of-headers sequence is detected. It then:

- Splits the first line into METHOD, PATH, and HTTP-VERSION
- Rejects unknown methods or versions with MalformedRequestError (triggers 400)
- Strips query strings from the path and maps bare "/" to "/index.html"
- Blocks path traversal attempts (".." in path) as a security measure
- Parses all remaining lines into a lowercase-keyed headers dictionary

1.5 HTTP response generation

The response builder solves the URI to a filesystem path inside static/ using os.path.realpath(), then applies the following decision chain:

Code	Trigger condition	Server behaviour
200 OK	File found and readable	full headers + body returned
304 Not Modified	If-Modified-Since >= file mtime	no body sent
400 Bad Request	Request line unparseable	connection closed immediately
403 Forbidden	File exists but owner-read bit (S_IRUSR) is not set	
404 Not Found	Requested file does not exist in static/ directory	

For 200 responses, the builder reads the file in binary mode, determines the MIME type from the file extension, calculates Content-Length from the byte count, and retrieves the file modification time for the Last-Modified header. For HEAD requests, the body bytes are omitted while all headers remain identical.

1.6 Caching — Last-Modified and If-Modified-Since

Every 200 OK response includes a Last-Modified header formatted as an RFC 2822 HTTP-date using Python's email.utils.formatdate(). When a client resends the request with an If-Modified-Since header, the server parses it with email.utils.parsedate_to_datetime() and compares it to the file's modification time. If the file has not been modified since that date, a 304 response with no body is returned. Malformed date strings are silently ignored, and a normal 200 is served instead.

1.7 Connection persistence

The Connection header is inspected after every request. The keep-alive decision follows a three-step priority chain:
If header is "close" → keep_alive = False
If header is "keep-alive" → keep_alive = True
Otherwise → keep_alive = (version == "HTTP/1.1") [RFC 7230 §6.3 default]

For persistent connections, the socket timeout is set to 10 seconds. If the client is idle beyond that window, the connection closes silently. Every response echoes the final Connection decision back to the client.

1.8 Request logging

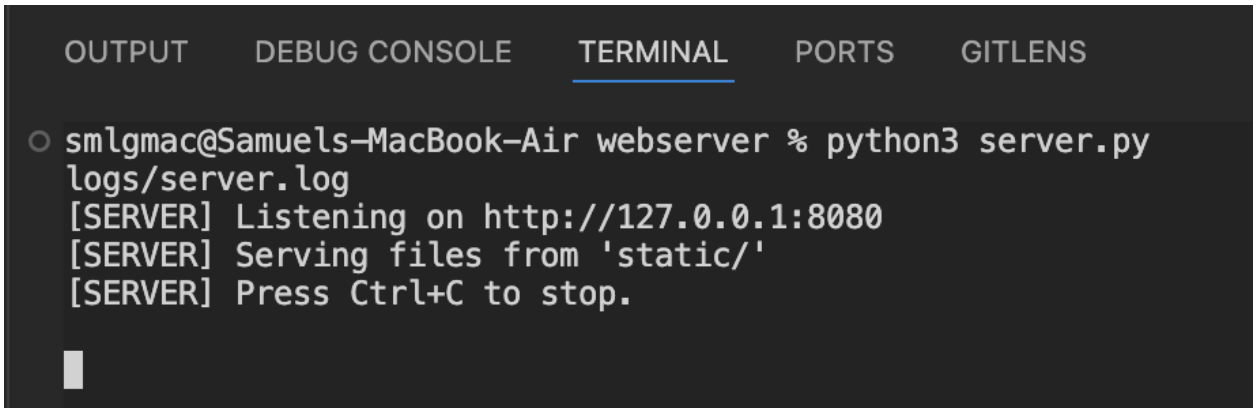
The ServerLogger class adds one tab-separated line per transaction to logs/server.log. A threading.Lock is acquired before every write to prevent concurrent threads from interleaving their log entries.

TIMESTAMP	CLIENT-IP	PATH	STATUS
2026-04-10 14:31:02	127.0.0.1	/index.html	200 OK
2026-04-10 14:31:05	127.0.0.1	/hello.txt	200 OK
2026-04-10 14:31:08	127.0.0.1	/missing.html	404 Not Found
2026-04-10 14:31:11	127.0.0.1	/hello.txt	304 Not Modified
2026-04-10 14:31:14	127.0.0.1	BAD_REQUEST	400 Bad Request
2026-04-10 14:31:17	127.0.0.1	/secret.html	403 Forbidden

2. Program Demonstration & Results

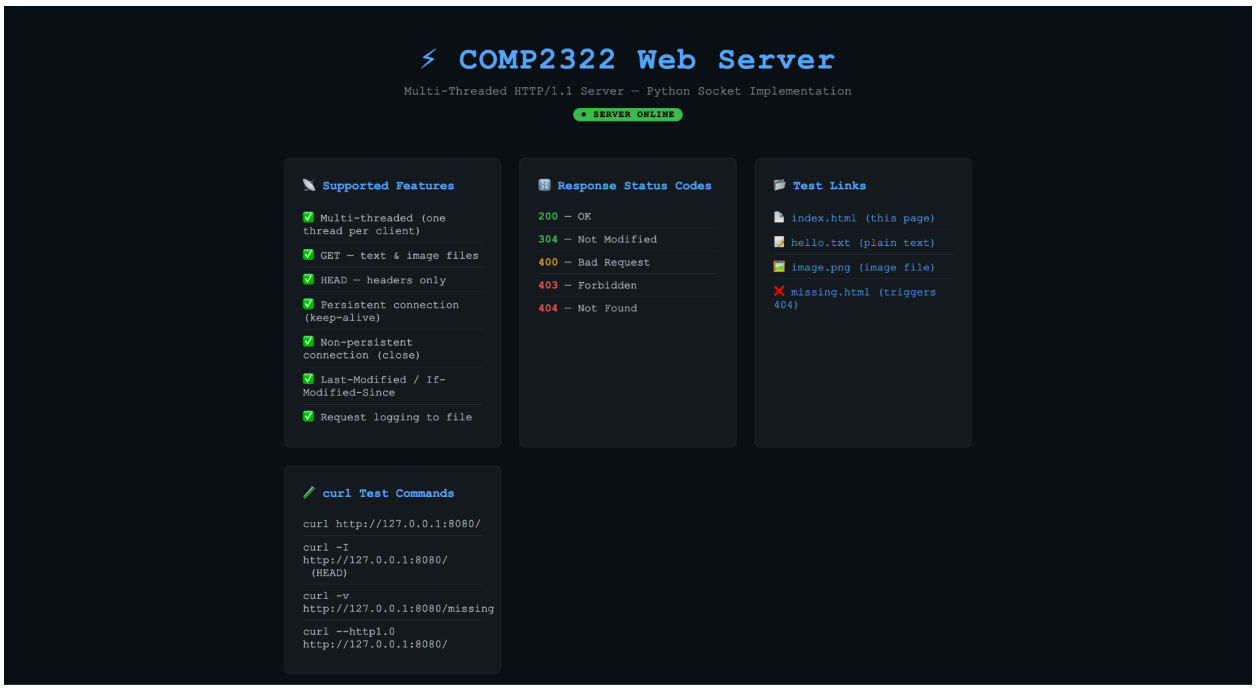
Screenshot 1 — Server startup

Start the server with: python3 server.py



Screenshot 2 — Browser homepage (200 OK)

Open http://127.0.0.1:8080 in a browser.



Screenshot 3 — GET text file (200 OK)

Run: curl -v http://127.0.0.1:8080/hello.txt

```
smlgmac@Samuels-MacBook-Air webserver % curl -v http://127.0.0.1:8080/hello.txt
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET /hello.txt HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 02 Apr 2026 07:39:29 GMT
< Server: Comp2322-WebServer/1.0
< Content-Type: text/plain; charset=utf-8
< Content-Length: 471
< Connection: keep-alive
< Last-Modified: Sun, 29 Mar 2026 09:11:21 GMT
<
Hello from the COMP2322 Multi-Threaded Web Server!

This is a plain text file served by our custom Python socket server.
The server correctly sets:

  Content-Type:   text/plain; charset=utf-8
  Content-Length: (auto-calculated)
  Last-Modified:  (file modification time in RFC 2822 format)
  Connection:     keep-alive   (or close if requested)

Try requesting this file again quickly — if the server sends 304 Not Modified,
the caching headers are working correctly!
* Connection #0 to host 127.0.0.1 left intact
```

Screenshot 4 — GET image file (200 OK)

Run: curl -o received.jpg http://127.0.0.1:8080/image.jpg && file received.jpg

```
smlgmac@Samuels-MacBook-Air webserver % curl -o received.jpg http://127.0.0.1:8080/image.jpeg && file received.jpg
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  8113  100  8113    0     0  4065k      0 --:--:-- --:--:-- --:--:-- 7922k
received.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 300x168, comp
onents 3
```

Screenshot 5 — HEAD request

Run: curl -I http://127.0.0.1:8080/index.html

```
● smlgmac@Samuels-MacBook-Air webserver % curl -I http://127.0.0.1:8080/index.html
  HTTP/1.1 200 OK
  Date: Thu, 02 Apr 2026 07:46:05 GMT
  Server: Comp2322-WebServer/1.0
  Content-Type: text/html; charset=utf-8
  Content-Length: 4262
  Connection: keep-alive
  Last-Modified: Sun, 29 Mar 2026 03:43:20 GMT
```

Screenshot 6 — 304 Not Modified

Run Step 1 (curl -I to get Last-Modified), then Step 2 (curl -v with If-Modified-Since).

```
● smlgmac@Samuels-MacBook-Air webserver % curl -I http://127.0.0.1:8080/hello.txt
  HTTP/1.1 200 OK
  Date: Thu, 02 Apr 2026 08:23:01 GMT
  Server: Comp2322-WebServer/1.0
  Content-Type: text/plain; charset=utf-8
  Content-Length: 471
  Connection: keep-alive
  Last-Modified: Sun, 29 Mar 2026 09:11:21 GMT

● smlgmac@Samuels-MacBook-Air webserver % curl -v -H "If-Modified-Since: Sun, 29 Mar 2026 09:11:21 GMT" \
      http://127.0.0.1:8080/hello.txt
  *   Trying 127.0.0.1:8080...
  * Connected to 127.0.0.1 (127.0.0.1) port 8080
  > GET /hello.txt HTTP/1.1
  > Host: 127.0.0.1:8080
  > User-Agent: curl/8.7.1
  > Accept: */*
  > If-Modified-Since: Sun, 29 Mar 2026 09:11:21 GMT
  >
  * Request completely sent off
  < HTTP/1.1 304 Not Modified
  < Date: Thu, 02 Apr 2026 08:23:50 GMT
  < Connection: keep-alive
  <
  * Connection #0 to host 127.0.0.1 left intact
```

Screenshot 7 — 400 Bad Request

Run: python3 -c "import socket; s=socket.socket(); s.connect(('127.0.0.1',8080)); s.send(b'NOTAVERB /path\r\n\r\n'); print(s.recv(1024).decode()); s.close()"

```
● smlgmac@Samuels-MacBook-Air webserver % python3 -c "import socket; s=socket.socket(); s.connect(('127.0.0.1',8080)); s.send(b'NOTAVER
  B /path\r\n\r\n'); print(s.recv(1024).decode()); s.close()"

  HTTP/1.1 400 Bad Request
  Date: Thu, 02 Apr 2026 07:52:31 GMT
  Server: Comp2322-WebServer/1.0
  Content-Type: text/html; charset=utf-8
  Content-Length: 590
  Connection: close
  <!DOCTYPE html>
  <html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>400 Bad Request</title>
    <style>
      body { font-family: monospace; background:#111; color:#eee;
             display:flex; flex-direction:column; align-items:center;
             justify-content:center; height:100vh; margin:0; }
      h1   { font-size:5rem; margin:0; color:#e55; }
      p    { font-size:1.2rem; color:#aaa; }
    </style>
  </head>
  <body>
    <h1>400</h1>
    <h2>Bad Request</h2>
    <p>The server could not understand your request.</p>
    <p><a href="/" style="color:#7af">← Back to Home</a></p>
  </body>
  </html>
```

Screenshot 8 — 403 Forbidden

Run: echo 'secret' > static/secret.html && chmod 000 static/secret.html

Then: ls -la static/secret.html to view permissions (show ---------- permissions)

Next: curl -v http://127.0.0.1:8080/secret.html

Restore after: chmod 644 static/secret.html

server.py | secret.html M | run_tests.sh

webserver > static > secret.html

You, 3 hours ago | 1 author (You)

```
1  secret      You, 3 hours ago • Uncommitted changes
2
```

OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
smlgmac@Samuels-MacBook-Air webserver % chmod 000 static/secret.html
curl -v http://127.0.0.1:8080/secret.html
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET /secret.html HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Date: Thu, 02 Apr 2026 12:32:52 GMT
< Server: Comp2322-WebServer/1.0
< Content-Type: text/html; charset=utf-8
< Content-Length: 592
< Connection: keep-alive
<
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>403 Forbidden</title>
  <style>
    body { font-family: monospace; background:#111; color:#eee;
           display:flex; flex-direction:column; align-items:center;
           justify-content:center; height:100vh; margin:0; }
    h1   { font-size:5rem; margin:0; color:#e55; }
    p    { font-size:1.2rem; color:#aaa; }
  </style>
</head>
<body>
  <h1>403</h1>
  <h2>Forbidden</h2>
  <p>You do not have permission to access this resource.</p>
  <p><a href="/" style="color:#7af">← Back to Home</a></p>
</body>
* Connection #0 to host 127.0.0.1 left intact
</html>%
```

Screenshot 9 — 404 Not Found

Run: curl -v http://127.0.0.1:8080/does-not-exist.html

Also open in browser to show custom 404 page rendered.

```
smlgmac@Samuels-MacBook-Air webserver % curl -v http://127.0.0.1:8080/does-not-exist.html
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET /does-not-exist.html HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< Date: Thu, 02 Apr 2026 08:28:35 GMT
< Server: Comp2322-WebServer/1.0
< Content-Type: text/html; charset=utf-8
< Content-Length: 850
< Connection: keep-alive
<
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>404 — Not Found</title>
  <style>
    body {
      background: #0d1117; color: #c9d1d9;
      font-family: 'Courier New', monospace;
      display: flex; flex-direction: column;
      align-items: center; justify-content: center;
      height: 100vh; margin: 0; text-align: center;
    }
    h1 { font-size: 8rem; margin: 0; color: #f85149; line-height: 1; }
    h2 { font-size: 1.5rem; color: #8b949e; margin: 1rem 0; }
    p  { color: #8b949e; max-width: 400px; }
    a  { color: #58a6ff; text-decoration: none; }
    a:hover { text-decoration: underline; }
  </style>
</head>
<body>
  <h1>404</h1>
  <h2>File Not Found</h2>
  <p>The file you requested does not exist on this server.</p>
  <p style="margin-top:2rem"><a href="/">← Return to Home</a></p>
</body>
</html>
* Connection #0 to host 127.0.0.1 left intact
```

Screenshot 10 — Connection keep-alive

Run: curl -v --keepalive http://127.0.0.1:8080/index.html http://127.0.0.1:8080/hello.txt

Capturing curl showing 'Re-using existing connection' and server terminal showing one CONNECT but multiple responses.

```
smlgmac@Samuels-MacBook-Air webserver % curl -v --keepalive http://127.0.0.1:8080/index.html http://127.0.0.1:8080/hello.txt

  <div class="card">
    <h2>🔪 Supported Features</h2>
    <ul>
      <li>✅ Multi-threaded (one thread per client)</li>
      <li>✅ GET - text &amp; image files</li>
      <li>✅ HEAD - headers only</li>
      <li>✅ Persistent connection (keep-alive)</li>
      <li>✅ Non-persistent connection (close)</li>
      <li>✅ Last-Modified / If-Modified-Since</li>
      <li>✅ Request logging to file</li>
    </ul>
  </div>

  <div class="card">
    <h2>🔢 Response Status Codes</h2>
    <ul>
      <li><span class="status ok">200</span> - OK</li>
      <li><span class="status ok">304</span> - Not Modified</li>
      <li><span class="status warn">400</span> - Bad Request</li>
      <li><span class="status err">403</span> - Forbidden</li>
      <li><span class="status err">404</span> - Not Found</li>
    </ul>
  </div>

  <div class="card links">
    <h2>📂 Test Links</h2>
    <a href="/index.html">📄 index.html (this page)</a>
    <a href="/hello.txt">📝 hello.txt (plain text)</a>
    <a href="/image.png">🖼️ image.png (image file)</a>
    <a href="/missing.html">❌ missing.html (triggers 404)</a>
  </div>

  <div class="card">
    <h2>🧪 curl Test Commands</h2>
    <ul>
      <li>curl http://127.0.0.1:8080/</li>
      <li>curl -I http://127.0.0.1:8080/  (HEAD)</li>
      <li>curl -v http://127.0.0.1:8080/missing</li>
      <li>curl --http1.0 http://127.0.0.1:8080/</li>
    </ul>
  </div>

</div>

<footer>
  <p>COMP2322 Computer Networking · PolyU · 2026</p>
</footer>

</body>
</html>
* Connection #0 to host 127.0.0.1 left intact
* Found bundle for host: 0xc550285a0 [serially]
* Can not multiplex, even if we wanted to
* Re-using existing connection with host 127.0.0.1
> GET /hello.txt HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 02 Apr 2026 08:29:35 GMT
< Server: Comp2322-WebServer/1.0
< Content-Type: text/plain; charset=utf-8
< Content-Length: 471
< Connection: keep-alive
< Last-Modified: Sun, 29 Mar 2026 09:11:21 GMT
<
Hello from the COMP2322 Multi-Threaded Web Server!

This is a plain text file served by our custom Python socket server.
The server correctly sets:

  Content-Type:   text/plain; charset=utf-8
  Content-Length: (auto-calculated)
  Last-Modified:  (file modification time in RFC 2822 format)
  Connection:     keep-alive   (or close if requested)

Try requesting this file again quickly - if the server sends 304 Not Modified,
the caching headers are working correctly!
* Connection #0 to host 127.0.0.1 left intact
smlgmac@Samuels-MacBook-Air webserver %
```

Screenshot 11 — Connection closed

Run: curl -v -H 'Connection: close' http://127.0.0.1:8080/index.html

Notice “Closing Connection” at the bottom of the picture

```
smlgmac@Samuels-MacBook-Air webserver % curl -v -H 'Connection: close' http://127.0.0.1:8080/index.html
    .ok     { color: var(--green);  }
    .warn   { color: var(--yellow); }
    .err    { color: var(--red);    }
    .links a {
      display: block;
      color: var(--accent);
      text-decoration: none;
      padding: 0.4rem 0;
      font-size: 0.9rem;
      border-bottom: 1px solid var(--border);
    }
    .links a:last-child { border-bottom: none; }
    .links a:hover { text-decoration: underline; }
    footer {
      margin-top: 4rem;
      color: var(--muted);
      font-size: 0.8rem;
      text-align: center;
    }
  </style>
</head>
<body>

<header>
  <h1>⚡  COMP2322 Web Server</h1>
  <p>Multi-Threaded HTTP/1.1 Server — Python Socket Implementation</p>
  <span class="badge">● SERVER ONLINE</span>
</header>

<div class="card-grid">

  <div class="card">
    <h2>🪶 Supported Features</h2>
    <ul>
      <li>✅ Multi-threaded (one thread per client)</li>
      <li>✅ GET — text &amp; image files</li>
      <li>✅ HEAD — headers only</li>
      <li>✅ Persistent connection (keep-alive)</li>
      <li>✅ Non-persistent connection (close)</li>
      <li>✅ Last-Modified / If-Modified-Since</li>
      <li>✅ Request logging to file</li>
    </ul>
  </div>

  <div class="card">
    <h2>🔢 Response Status Codes</h2>
    <ul>
      <li><span class="status ok">200</span> — OK</li>
      <li><span class="status ok">304</span> — Not Modified</li>
      <li><span class="status warn">400</span> — Bad Request</li>
      <li><span class="status err">403</span> — Forbidden</li>
      <li><span class="status err">404</span> — Not Found</li>
    </ul>
  </div>

  <div class="card links">
    <h2>📂 Test Links</h2>
    <a href="/index.html">📄 index.html (this page)</a>
    <a href="/hello.txt">📝 hello.txt (plain text)</a>
    <a href="/image.png">🖼️ image.png (image file)</a>
    <a href="/missing.html">❌ missing.html (triggers 404)</a>
  </div>

  <div class="card">
    <h2>🧪 curl Test Commands</h2>
    <ul>
      <li>curl http://127.0.0.1:8080/</li>
      <li>curl -I http://127.0.0.1:8080/  (HEAD)</li>
      <li>curl -v http://127.0.0.1:8080/missing</li>
      <li>curl --http1.0 http://127.0.0.1:8080/</li>
    </ul>
  </div>

</div>

<footer>
  <p>COMP2322 Computer Networking · PolyU · 2026</p>
</footer>

</body>
</html>
* Closing connection
```

Screenshot 12 — Multi-threading (concurrent clients)

Run: for i in {1..10}; do curl -s http://127.0.0.1:8080/index.html > /dev/null & done; wait

```
smlgmac@Samuels-MacBook-Air webserver % for i in {1..10}; do curl -s http://127.0.0.1:8080/index.html >
/dev/null & done; wait
[2] 60993
[3] 60994
[4] 60995
[5] 60996
[6] 60997
[7] 60998
[8] 60999
[9] 61000
[10] 61001
[11] 61002
[5]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[3]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[6]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[7]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[4]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[8]    done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[11]  + done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[10]  + done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[2]  - done       curl -s http://127.0.0.1:8080/index.html > /dev/null
[9]  + done       curl -s http://127.0.0.1:8080/index.html > /dev/null
```

Capturing server terminal showing interleaved [CONNECT] and [200 OK] lines proves parallel execution.

```
[CONNECT] New connection from 127.0.0.1:49407
[CONNECT] New connection from 127.0.0.1:49406
[CONNECT] New connection from 127.0.0.1:49405
[CONNECT] New connection from 127.0.0.1:49408
[CONNECT] New connection from 127.0.0.1:49409
[CONNECT] New connection from 127.0.0.1:49410
[CONNECT] New connection from 127.0.0.1:49411
[CONNECT] New connection from 127.0.0.1:49412
[CONNECT] New connection from 127.0.0.1:49413
[CONNECT] New connection from 127.0.0.1:49414
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
[200 OK] GET /index.html — 127.0.0.1 (keep-alive)
```

3. Server Log File

The log file is automatically created at logs/server.log on first server start. It records one entry per HTTP transaction with four tab-separated fields: timestamp, client IP, requested path, and response status.

Log format:

```
TIMESTAMP                CLIENT-IP      PATH                    STATUS
```

Run: cat logs/server.log after completing all test scenarios.
Example format of the server.log file:

```
server.py    server.log M    run_tests.sh
webserver > logs > server.log
You, 39 minutes ago | 1 author (You)
# Comp 2322 Web Server — Request Log
# Format: TIMESTAMP  CLIENT-IP  PATH  STATUS
# Started: 2026-03-29 01:58:01.224348
#
TIMESTAMP           CLIENT-IP   PATH            STATUS
------------------------------------------------------------------------------------------
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK
2026-03-30 17:18:50 127.0.0.1   /index.html 200 OK      You, 2 days ago • Testing sce
2026-03-30 17:19:22 127.0.0.1   /index.html 200 OK
2026-03-30 17:20:08 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:20:58 127.0.0.1   /image.jpeg 200 OK
2026-03-30 17:22:54 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:22:54 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:23:52 127.0.0.1   /index.html 200 OK
2026-03-30 17:24:27 127.0.0.1   BAD_REQUEST 400 Bad Request
2026-03-30 17:26:14 127.0.0.1   /secret.html    403 Forbidden
2026-03-30 17:26:42 127.0.0.1   /this-file-does-not-exist.html  404 Not Found
2026-03-30 17:27:29 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:27:40 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:28:00 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:28:58 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:29:26 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:29:45 127.0.0.1   /index.html 200 OK
2026-03-30 17:30:50 127.0.0.1   /hello.txt  304 Not Modified
2026-03-30 17:31:27 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:31:35 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:32:04 127.0.0.1   /index.html 200 OK
2026-03-30 17:32:04 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:32:04 127.0.0.1   /index.html 200 OK
2026-03-30 17:38:59 127.0.0.1   /index.html 200 OK
2026-03-30 17:39:13 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:41:36 127.0.0.1   /this-does-not-exist.html   404 Not Found
2026-03-30 17:41:36 127.0.0.1   /secret.html    403 Forbidden
2026-03-30 17:41:36 127.0.0.1   /hello.txt  304 Not Modified
2026-03-30 17:41:36 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:41:36 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:41:36 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /index.html 200 OK
2026-03-30 17:41:36 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:43:01 127.0.0.1   /hello.txt  200 OK
2026-03-30 17:43:01 127.0.0.1   /hello.txt  200 OK
```

Implementation notes:

- Thread safety is ensured by threading.Lock acquired before every file write.
- Directories are created automatically with os.makedirs(exist_ok=True) before first use.
- For malformed requests, the path field is recorded as "BAD_REQUEST" since no valid path exists.
- Timestamps use Python's datetime.datetime.now() in local time with second precision.

4. README File

The README.md file is included in the repository root and contains all information required for a new user to install, run, and test the server without reading the source code.

4.1 README.md contents summary:

Project description — what the server is and what it does
Feature list — all implemented HTTP features in a table
Requirements — Python 3.8+ (no third-party packages)
Installation — git clone and directory structure explanation
How to run — python3 server.py with optional port override
Testing with curl — all test commands for every status code
Log file format — column descriptions
Security notes — path traversal and file permission handling

Excerpt — key run commands from README.md:

```
# Start the server (default port 8080)
python3 server.py
# Override port
python3 server.py 9090
# Test all status codes
curl http://127.0.0.1:8080/index.html                          # 200
curl -H 'If-Modified-Since: Fri, 31 Dec 2099 23:59:59 GMT' \  # 304
     http://127.0.0.1:8080/hello.txt
curl http://127.0.0.1:8080/missing.html                        # 404
chmod 000 static/secret.html                                   # 403
curl http://127.0.0.1:8080/secret.html
```

Screenshot — README.md rendered on GitHub (See next page):

SWL0120 / Multi-Thread-Web-Server

Code | Issues | Pull requests | Actions | Projects | Wiki | Security and quality | Insights | Settings

Multi-Thread-Web-Server Public

main | 2 branches | 0 tags | Go to file | Add file | Code

SWL0120 Merge pull request #1 from SWL0120/testing · 8ec

webserver	Post-fixing testing	yesterday
.DS_Store	Removing unwanted cases	3 days ago
README.md	Update project tree format in README	3 days ago
note - revisions.txt	Testing specific cases	3 days ago

README

COMP2322 — Multi-Threaded HTTP Web Server

A lightweight HTTP/1.1 web server built from scratch using Python's `socket` module.
No `http.server`, no `Flask`/`Django` — pure socket programming.

Features

Feature	Details
Multi-threading	Each client connection runs in its own daemon thread
Methods	GET (full + images) and HEAD
Status codes	200 OK, 304 Not Modified, 400 Bad Request, 403 Forbidden, 404 Not Found
Caching headers	Last-Modified (mtime) and If-Modified-Since (parsed)
Persistence	Connection: keep-alive and Connection: close
Logging	One tab-separated line per request appended to logs/server.log
MIME types	.html, .htm, .txt, .js, .jpg, .png, .gif, .ico, .webp, …

Requirements

- Python 3.8+ (no third-party packages required — standard library only)
- A terminal / command prompt

Check your Python version:

```
python3 --version
```

Project Structure

```
webserver/
├── server.py              - Entry point — run this file
├── utils/
│   ├── __init__.py
│   ├── request_parser.py  - Parses raw HTTP request bytes
│   ├── response_builder.py - Builds HTTP response bytes
│   └── logger.py          - Thread-safe request logger
├── static/                - Files served in /static/
│   ├── index.html
│   ├── hello.txt
│   ├── 404.html
│   └── image.jpg          - Add your own images here
├── logs/
│   └── server.log         - Auto-created on first run
└── README.md
```

How to Run

1. Clone / download the project

```
git clone https://github.com/yourusername/comp2322-webserver.git
cd comp2322 webserver
```

2. Start the server (default port 8080)

```
python3 server.py
```

3. Override the port (optional)

```
python3 server.py 9090
```

The server prints:

```
[STARTED] Listening on http://127.0.0.1:8080
[SERVER] Serving files from 'static/'
[STARTED] Press Ctrl+C to stop.
```

4. Open your browser

Navigate to: http://127.0.0.1:8080

Testing with curl

Basic GET request

```
curl http://127.0.0.1:8080/index.html
```

HEAD request (headers only, no body)

```
curl -I http://127.0.0.1:8080/index.html
```

Verbose output (see all headers)

```
curl -v http://127.0.0.1:8080/hello.txt
```

404 Not Found

```
curl -v http://127.0.0.1:8080/does-not-exist.html
```

304 Not Modified (conditional GET)

```
# First, get the Last-Modified date
curl -I http://127.0.0.1:8080/index.html

# Then send it back — should return 304 Not Modified
curl -v -H "If-Modified-Since: Wed, 01 Jan 2025 00:00:00 GMT" \
     http://127.0.0.1:8080/index.html
```

Non-persistent connection (HTTP/1.0 style)

```
curl --http1.0 http://127.0.0.1:8080/index.html
```

Explicit Connection: close

```
curl -v -H "Connection: close" http://127.0.0.1:8080/index.html
```

Multiple simultaneous clients (stress test)

```
for i in {1..10}; do
    curl -s http://127.0.0.1:8080/index.html > /dev/null &
done
wait
echo "All done."
```

Log File Format

`logs/server.log` is appended with one entry per request:

```
TIMESTAMP           CLIENT-IP   PATH        STATUS

2026-04-20 14:32:01 127.0.0.1   /index.html 200 OK
2026-04-20 14:32:02 127.0.0.1   /hello.txt  200 OK
2026-04-20 14:32:03 127.0.0.1   /missing.html 404 Not Found
2026-04-20 14:32:15 127.0.0.1   /index.html 304 Not Modified
```

Adding Files to Serve

Place any file inside the `static/` directory and it becomes immediately accessible:

```
cp ~/my-photo.jpg static/
# Now accessible at: http://127.0.0.1:8080/my-photo.jpg
```

Security Notes

- **Path traversal:** requests containing `..` are rejected with 403.
- **Forbidden files:** files without read permission return 403.
- **Static directory only:** files outside `static/` are inaccessible.

Author

- Name: Samuel Ho
- Student ID: 25142247X
- Course: COMP2322 Computer Networking, PolyU
- Due: Apr 26, 2026

About

For COMP2322 Final Project

Readme | Activity | 0 stars | 0 watching | 0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

SWL0120 Sam

Languages

Python 93.8% | Shell 3.0% | HTML 3.2%

Suggested workflows

Based on your tech stack

Pylint — Configure
Lint a Python application with Pylint.

Python application — Configure
Test and lint Python applications.

ELSA Semantic generator — Configure
Generate ELSA semantic for web crawling in your workflows.

More workflows | Explore GitHub Marketplace

© 2026 GitHub Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

5. Grading Rubric Self-Assessment

The following table maps each rubric criterion to the implementation evidence in this report.

Criterion	Marks	Status
Multi-threaded server (one thread per connection)	**5 marks**	**Implemented**
Proper request/response message exchange	**5 marks**	**Implemented**
GET — text files	**5 marks**	**Implemented**
GET — image files	**5 marks**	**Implemented**
HEAD command	**5 marks**	**Implemented**
200 OK	**5 marks**	**Implemented**
400 Bad Request	**5 marks**	**Implemented**
403 Forbidden	**5 marks**	**Implemented**
404 Not Found	**5 marks**	**Implemented**
304 Not Modified	**5 marks**	**Implemented**
Last-Modified header	**5 marks**	**Implemented**
If-Modified-Since header	**5 marks**	**Implemented**
Connection: keep-alive	**5 marks**	**Implemented**
Connection: close	**5 marks**	**Implemented**
Design & implementation summary (report)	**10 marks**	**Implemented**
Demonstration with screenshots (report)	**5 marks**	**Implemented**
Complete log file (report)	**5 marks**	**Implemented**
Clear README file (report)	**5 marks**	**Implemented**
Source code quality (comments, naming, no runtime errors)	**5 marks**	**Implemented**

GitHub repository: https://github.com/SMLG120/Multi-thread-Web-Server